

変更するために、構造を読む

利用者の比較 → 画面・サービス・核 → 担当ファイル → 主要な機能

この資料は実装の地図である。式の導出や細かな入力仕様は専用資料へ送り、どこを読むか・変更がどこへ届くかを短い経路で示す。

全体の問い

何がどこにあるか
誰が誰を使うか
草案、固定結果、表示、計算、保存を見分ける。

一つの枝へ降りる

画面 / サービス
場 / 風統計 / 一飛行
標本集合 / 出力
枝の責務、入出力、一段下の実体を見る。

変更と検証へ戻る

変更の逆引き
親の目的と横の契約を戻り先にする。全関数を同時に覚えなくてよい。

役割を色で追う

青：操作の入口 緑：環境場
橙：飛行・物理・積分 紫：結果と保存
灰：試験・資料・管理
色だけに意味を持たせず、実パスと責務を併記する。

線と箱を読み分ける

細線：実体の包含。実線矢印：主な呼出。
灰破線：import 依存。青点線：値の受渡し。
四角：実ファイル/クラス。丸角：機能。
図は静的な全呼出網ではなく、その枝を理解するための主要な関係を選ぶ。

実装と互換入口を分ける

0.24 で責務を実パッケージへ分けた。旧 `weather.py/dynamics.py` は公開名を保つ再 export 入口で、別実装を持たない。未実装の慣性・熱・気象誤差を、現役の仮感度集合へ混ぜない。

地図の次に読む資料

IMPLEMENTATION_NOTES.md：処理/入出力の詳細
THEORY_GUIDE.pdf：式・仮定・数値法
COMMANDS.md：実行と再現
IMPLEMENTATION_INDEX.json：実定義・公開名・静的依存

文書改訂 0.59・実装 0.58。固定 PR31 C2 95f09d2fc0ae を基準に、気象画面の担当と参照先を局所訂正した。コードと科学式は変更しない。0.58 の年間 48 区分、年間値の再利用、気圧面の操作枠は継承する。全校の科学的再検証ではなく、既存の受入・未確認は S36 へ戻る。式は THEORY、main への統合は別判断とする。

比較する操作を、実際の枝へ結ぶ

階層 0 / repo → frontend・backend・balloon_sim

草案を編集しても固定結果を変えない。そのため、画面の選択、計算の受入、物理の意味を別の実体に置く。

private repo : 同じ commit から起動する正本

frontend/ と backend/ を通常の Git 差分で育て、既存 balloon_sim/ と保存 fixture を同じ版へ結ぶ。

frontend/

React/TypeScript と継承した画面素材。草案・固定結果の選択、地図、履歴、気象画面。Vite が開発/ビルドを担う。

backend/

FastAPI と application service。
固定入力・一飛行・結果・草案。
weather/ : 取得。ensemble/ : 固定集合。climate/ : 平均・原標本の風統計。

balloon_sim/

environment : 保存場
flight : 一飛行
ensemble : 抽選・共通集計
results : 一飛行の出力。

tests / docs / tools

守る契約、構造/理論/操作の説明、生成/点検。計算の実行依存へ混ぜない。

references / examples

保存 GFS・JRA 原本と機体入力例。現在予報や実機係数の採用値として扱わない。

repo 外の利用状態

草案/仕事・完成場・固定結果は BALLOON_DATA_DIR へ。外生 DB/原標本 manifest の読取元は起動設定で別に指定。Git ソースや dist と別の寿命。

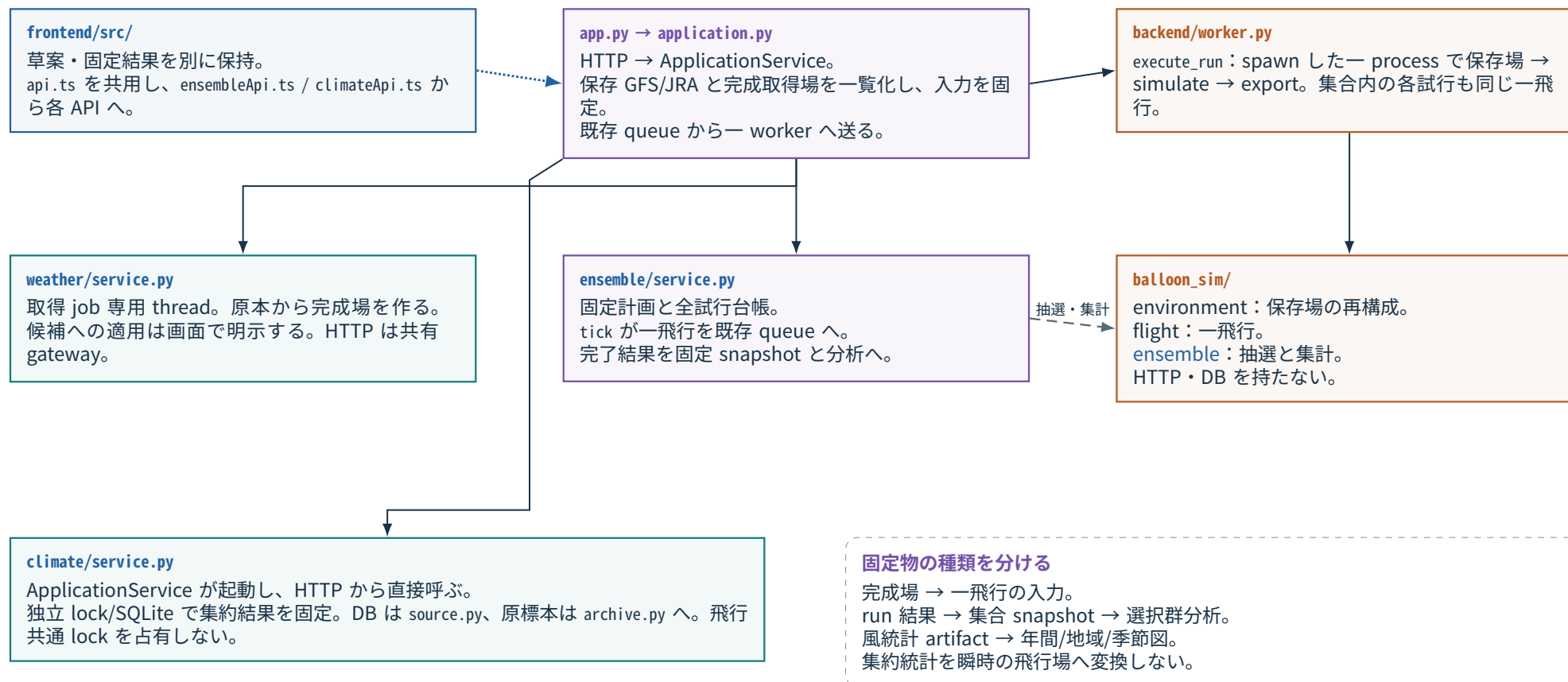
この分担が守ること

画面を編集しても保存結果の入力が変わらず、HTTP や DB を替えても物理核へ依存を逆流させない。表示・入力・場・数値の意味は別の枝へ戻れる。フォルダーを増やすことが目的ではない。

取得・飛行・風統計を、別の枝でつなぐ

階層 1 / frontend → application → weather・ensemble・climate・一飛行 worker

集合は既存の一飛行を同じ queue へ投入する。風統計は独立した集約/保存 service へ渡し、飛行 worker へは送らない。



frontend：共通の計画と、用途ごとの画面

階層 2 / frontend → src → App・状態・screens

育てた予報・詳細・気象・季節の画面を、同じ計画と固定結果へ結ぶ。React の共通状態と、各画面の描画状態は異なる枝にある。

frontend/src/：main.tsx から起動

main.tsx が React と Leaflet CSS を読み込む。App.tsx が共通ヘッダー・用途切替・保存入口を構成し、platform.css がその配置を担当する。

App.tsx

計画/気象/季節の行先を選ぶ。
候補の追加・延期・独立化を共通計画へ反映。画面の変更通知を受け、用途別の表示状態を保存側へ渡す。

useWorkspace.ts workspaceState.ts

草案・固定 run・選択・仕事・保存。
固定要求・復旧の枝へ。
jobObservation.ts へ単便の GET 監視を委譲。取得 UI も共有する。
集合は useEnsembleWorkspace へ。表示選択 token を共有。

api.ts

相対/api/v1 の JSON 送受信。
readRequest：期限付き GET。
request：一般送受信。集合/風統計の専用 API も利用する。backend へ。

条件・結果/操作

表示状態/季節への引渡し

screens/ForecastScreen.tsx

予報・詳細・季節計画を同じ画面素材で開く。
実結果と現在 source を projection へ渡し runtime へ同期。未読時も保存した群/時刻を保持。専用 portal へ ConfigEditor・SamplingEditor・固定結果選択・WeatherPreparation を載せる。
取得準備と入力編集は専用の枝へ降りる。

screens/WeatherScreen.tsx

人工の四画面と、実風統計の年間/地域/季節図を選ぶ。
実側の useClimateWorkspace が草案 query/固定 artifact、runtime が表示を担う。
人工の提案を保持し、実側は原日時の関心を App へ。統計と共通図の枝へ。

横で共有する型・入力

domain.ts：草案・固定結果、気象の内容照合、JST/UTC の秒を保つ変換。
fixedResultRead.ts：読取状態・表示名・詳細待機の判定。各 hook と画面が共有し、HTTP を持たない。
ConfigEditor.tsx：草案入力。固定結果は不変。

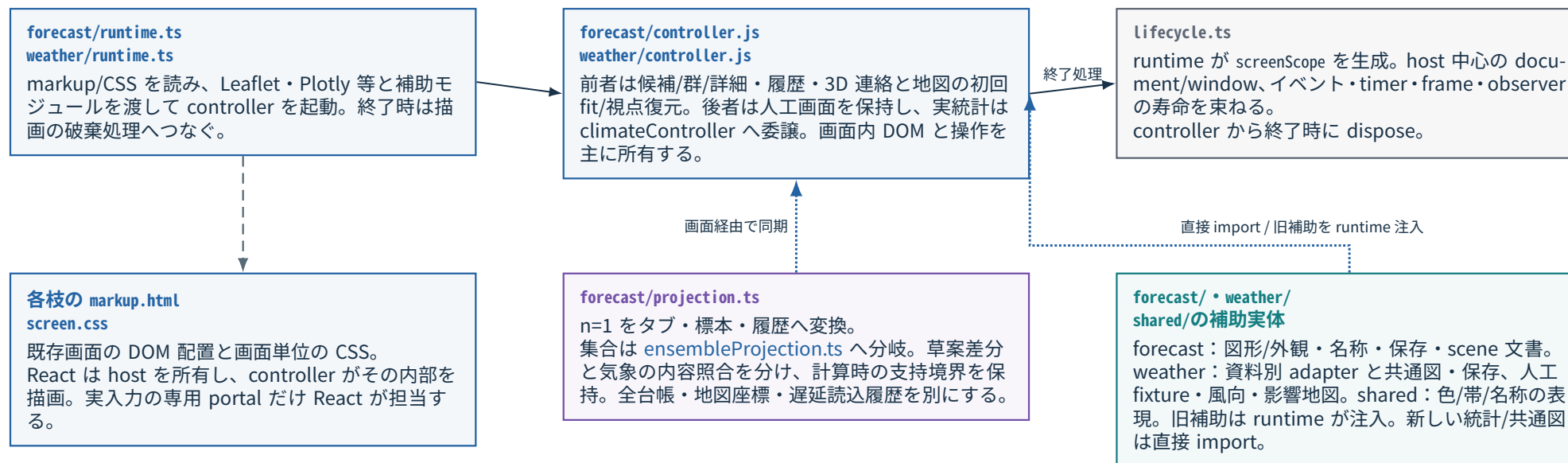
もう一段下へ降りる

runtime と controller の枝へ。
画面の共通時刻・群選択・地図視点は各 controller 側。App が全描画状態を直接操作する構成ではない。境界の詳細は実装ノートへ。

screens：画面素材を、限定した描画領域へ載せる

階層 3 / screens → forecast・weather・shared・lifecycle

各 React 画面が host を持ち、runtime が既存の画面と描画処理を載せる。画面の内容を一から描き直した構成ではなく、具体素材を取り込んだ移行途中の構成である。



3D の表示と読取りの枝へ

controller が scene-document.js で対象の固定 snapshot を作り、postMessage で /scene3d/index.html へ渡す。表示設定は親へ戻し、KML と保存に共有。配信元は frontend/scene3d/。src/main.js が地表 overlay・空中 route・hover の寿命を束ねる。Vite が npm の Cesium と固定高度格子を dist へ置く。

現在の構造で残る制約

大きな controller に複数の操作・描画責務が残り、型のない bridge や scope を介す。移行だけで分割完了とはしない。旧 MapPanel.tsx/HistoryPanel.tsx は残るが、現在 App の表示経路からは呼ばれていない。

図は読取時点の実ソースの関係。人工表示、実 n=1/仮感度集合、実 DB の期間統計、保存 JRA の元日時場を区別する。任意過去窓の取得 UI・代表性を定めた多年季節 MC は未実装。登録済み原日時の有限比較は原日時の枝へ。controller の寿命・操作全体・写真 3D 接続の受入を、この配置図だけで認定しない。

scene3d：同じ経路を、地形との関係へ開く

階層 4 / frontend → scene3d → src と data

写真地図の接続、固定結果の図形、地物の読取りは別の寿命を持つ。空中線を追加しても地表ホバーを壊さず、表示変更だけで Google へ接続し直さない。

index.html → src/main.js：表示の host

親の forecast/controller から固定 snapshot と表示設定を受ける。provider.js へ明示接続を渡し、候補 catalog、地表 overlay、空中 route、視点・読取りを束ねる。表示の変更通知は親の保存 UI/KML へ戻す。

src/protocol.js

候補/結果/選択群と geometry を検査。
地表線・空中線・幕の経緯度/相を対応づけ、ASL/人工 AGL を区別する。
生成元は親の scene-document。
allObjects は KML 用、3D は可視対象。

地表示の表示

src/overlays.js
点・帯・領域・地表線の Cesium 定義。
src/overlay-store.js
新定義を準備して差替え、失敗時は旧 source へ戻す。

src/route-layer.js

空中表示と高度準備の寿命。
実 ASL は地物照会なしで全経路。人工 AGL だけ有限照会・取消を持つ。
接続/視点は生成しない。
checkpoint が旧高度と表示を保持。

地物を読む補助群

surface-session / surface-pick
読取り順と通常描画待ち。
hover-controller / hover-labels
静止と名称、帯は band-readout。
表示中/退役待ち図形を地物から除外。

src/route-geometry.js

原経路の間に表示節点を追加。
同じ節点と曲率から空中線・連続幕を作る。
幕の地下余長は写真 mesh に隠れる。
空中要素は透過 pass とし、背後の地物深度を読めるようにする。

src/vertical-datum.js

表示専用の EGM96 格子補間。
data/WW15MGH.DAC
固定 15 分格子。data/README に出典。
route-layer がローカル fetch。
科学 ASL・KML 値を変更しない。

どこへ戻って判断するか

高度の意味と表示限界は THEORY、切替・再接続・保留の操作は COMMANDS へ。幕は上下の対応を示す。地形高の取得可否で実経路を分断せず、描画の重なりを衝突解析へ読み替えない。新しい高さ表示のために飛行核や保存結果の schema を変更しない。視点を動かす操作は main と camera-fit/view-memory へ戻る。

weather：資料を分け、図と保存を共有する

階層 3 / WeatherScreen → 状態・資料 adapter → 図別 controller・plots・export

人工標本と実集約値では読める量が違う。実側は固定結果内の区切りを選ぶ。原標本の 48 区分は年間専用。月中央値/幅、補助の平均、代表点風配はそれぞれの意味を保つ。

screens/WeatherScreen.tsx : FixtureWeatherScreen / ClimateWeatherScreen

人工側は既存 runtime/状態を接続。実側は hook の artifact を粒度別 VM へ変換し、React の query 入力と runtime.sync をつなぐ。用途切替と保存計画は App と共用。関心の受渡しは ClimateFlightHandoff.tsx へ。

useClimateWorkspace.ts

草案 query と固定 artifact を分ける。気圧面/風配点の探索を同じ最新要求列へ。再送 ID・応答寿命・GET 復帰を保持。入力/支持は climateDomain.ts。参照は weather_real へ保存。

climateApi.ts → api.ts

sources / create / get。
固定統計の API へ JSON 送受信。
集計条件の適用は POST、保存図の GET 再読は再集計しない。

weather/climateController.js

図別区切りと元格子点を選ぶ。年間だけ 48 区分を追加し、保存図の切替は POST なし。面操作は slider・選択/表示値・状況の枠を分ける。人工側は controller.js。

固定 summary

weather/sources/

fixtureSummary.ts：人工原標本の aggregateFixture → VM。
climateSummary.ts：固定 summary → 粒度別 VM。年間 48 区分の暦・件数を独立検査。
原標本分位の検査は../monthlyProfiles.ts を参照。旧 artifact の図も保つ。

weather/viewModel.ts

WindViewModel：時期・面・格子、平均量、母数、定義と固定来歴。分位と風配は別の値/count/階級を保持。
levelEdges / niceSpeedMax：表示区画/尺度。

weather/：図の意味ごとの担当

plots.ts：年間/地域。climateDistributions.ts：平均/風配・静穏と元格子の選択図。
monthlyProfiles.ts：固定 12 か月の中央値/幅・共通軸。平均から分位を作らない。

weather/export.ts

保存開始時の図/条件/来歴を固定し背景を埋める。
年間/地域・平均/風配・月分位の図を保存。

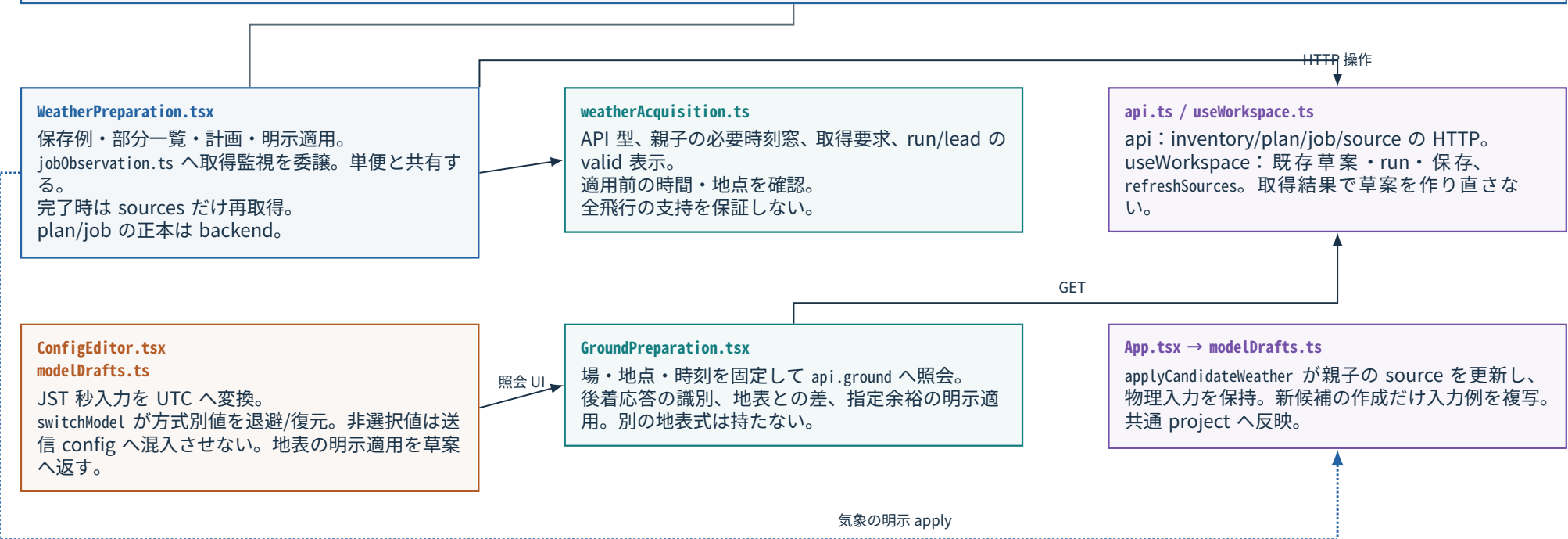
気象の準備：保存場の選択と、GFS の取得

階層 3 / ForecastScreen → 保存場・取得 UI・入力操作

同じ画面の共通準備と条件棚を使う。取得完了で増えるのは利用可能な場であり、草案や表示中の固定結果を自動で置換しない。

screens/ForecastScreen.tsx：既存画面と React の接点

##realWeather portal へ取得準備を載せる。bridge.acquisitionPreview() で controller へ丸め後の橙破線境界・run・必要窓・古化を渡す。固定結果の灰色支持境界は別に保持する。



保存先と、別の寿命を持つもの

ui_state.forecast_real.weatherPreparation: フォーム、手順位置、plan/job/request 参照、preview。editor_models: 非選択方式の編集値。固定 run は別参照。実風統計は別の weather_real 状態と固定 artifact へ。原日時試算は season_real の別状態へ。取得 GFS や半月統計を原日時場へ読み替えない。

集合の画面：固定結果を、選んで掘り下げる

階層 3 / ForecastScreen → 入力・非同期状態・描画 adapter

既存の候補・延期列・地図・詳細画面へ集合を接続する。仕事の進捗と表示中の snapshot を別に保持し、完了を自動表示しない。

SamplingEditor.tsx

仮幅/理由/N/seed、対応候補。
固定計画の確認 → 明示実行。
ensembleDomain.ts が型・元量と方式・親子計画・固定参照を束ねる。

useEnsembleWorkspace.ts

App が呼び、共通画面の寿命で保持。
計画・job・case・分析・履歴の cache と寿命。
case 読取状態と GET 再読。内部の
createEnsembleObserver が GET 観測/失敗後の待機を管理。
ensembleApi.ts で GET 期限/abort を接続。

ForecastScreen.tsx

既存 portal へ入力を載せる。
固定 case を collectionResult で変換し、現在
source との照合は別に加えてタブへ同期。
bridge で群分析/個別履歴を要求。

forecast/ensembleProjection.ts

collectionResult：全台帳と地図点を分離。
気象の由来照合は domain.ts と共用。
empiricalContours / analysisRoutes / aggregateTraces：共通分析の幾何・相別線/帯を描画値へ。
pairedRows：draw_id で対応を結ぶ。

forecast/controller.js → math.js

controller：標本選択・図・3D 連絡。math は領域所属判定。
analysisLoadState.ts：全 ID 一致の既存分析を再利用し、部分群の待機/失敗を保持。
未送信だけ担当取得後に再開。
historyLoadQueue.ts：集計から独立した原履歴 GET を画面内 4 並列で読み、未開始の旧選択を破棄する。

保存する参照、別に残る状態

Project.compare_results：固定 run/case。
ui_state.forecast_real.selectionContext：群の分析 ID/key。再読は getAnalysis へ。
計画保存は useWorkspace が担当する。

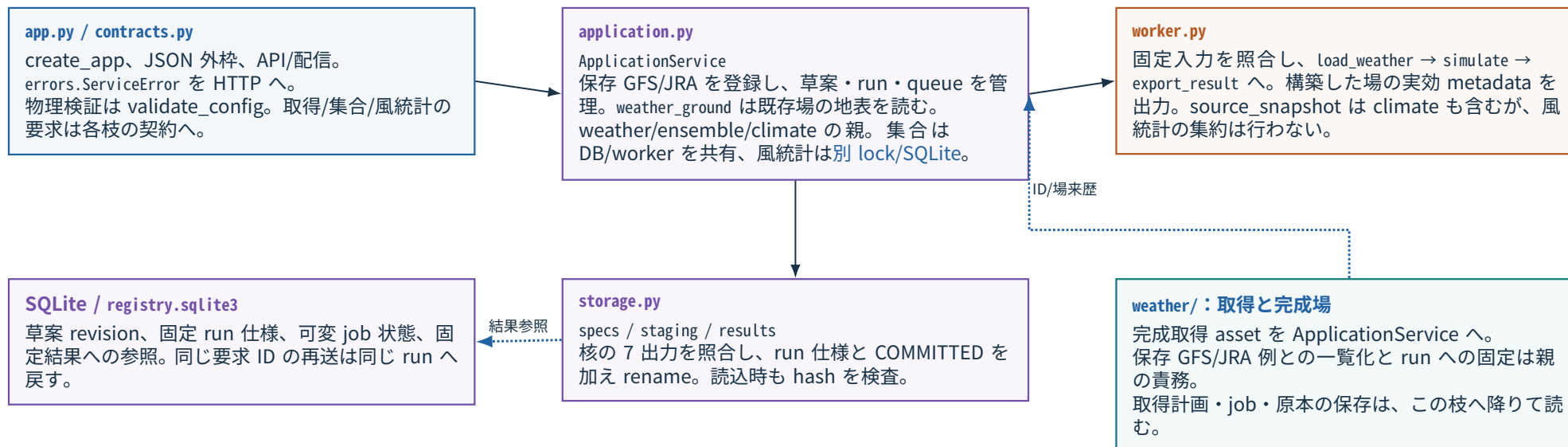
実装上の境界

地図点がない停止を全台帳から消さない。
禁止域の所属は frontend 判定であり、backend による幾何の独立再検証ではない。
renderer は既存 2D/3D を使う。

backend : 仕事と固定結果の寿命を管理する

階層 2 / backend → HTTP • application • worker • storage • weather • ensemble • climate

ApplicationService は単一利用者・一保存先を所有する。計算結果と完成した気象 asset を、途中の directory から区別して受け渡す。



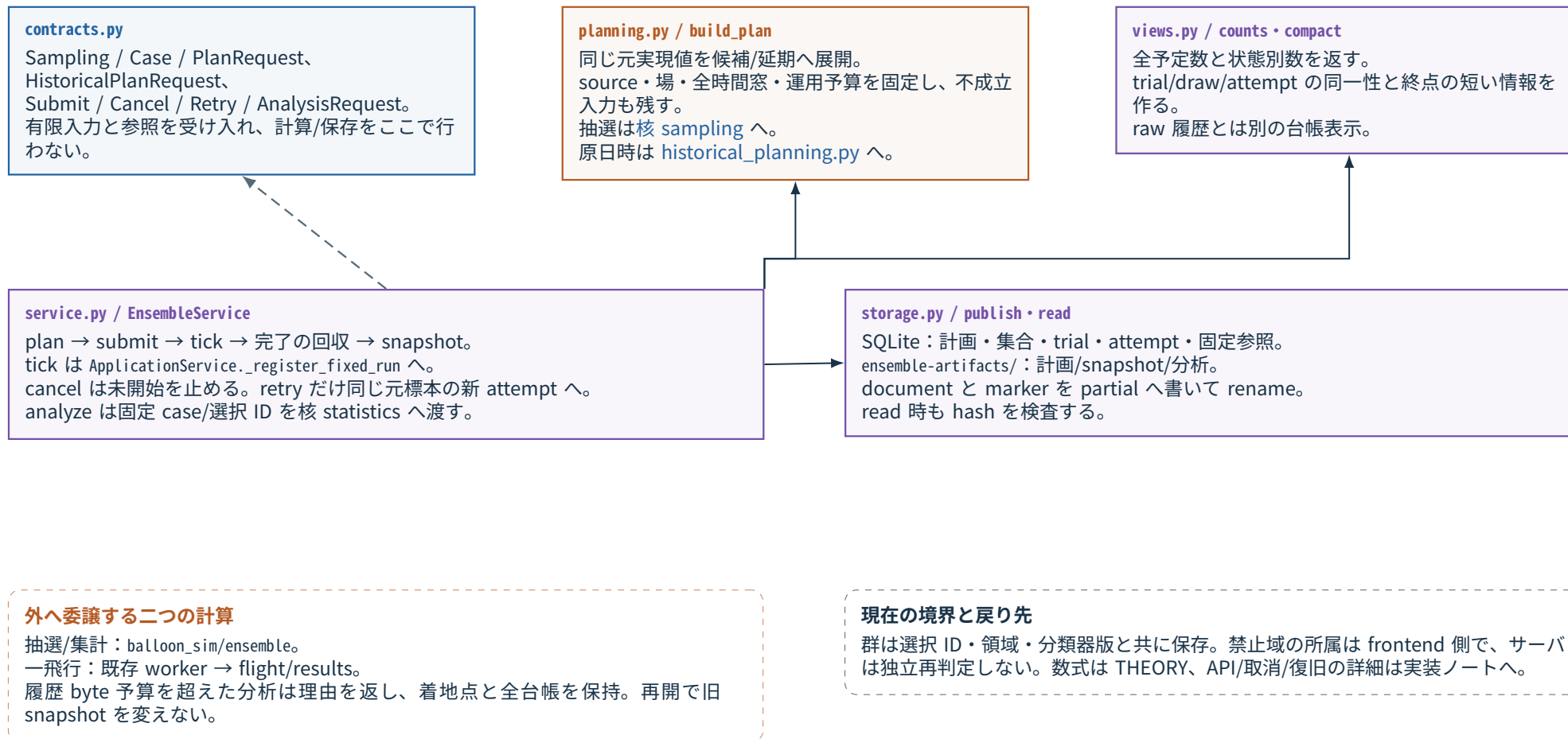
仕事の種類と、読取りの経路を分ける

飛行 job は待機中だけ取消、実行中は完了を待つ。取得は協調取消と明示 retry。集合は **ensemble/** へ。地表照会は app → application → load_weather → ground_altitude の読取経路で、飛行 queue へ送らない。起動 source 変更と旧結果の閲覧は別。風統計は HTTP から独立 service へ渡す。

ensemble : 全試行台帳から、不変の集合へ

階層 3 / backend → ensemble → 契約・計画・service・保存・表示用台帳

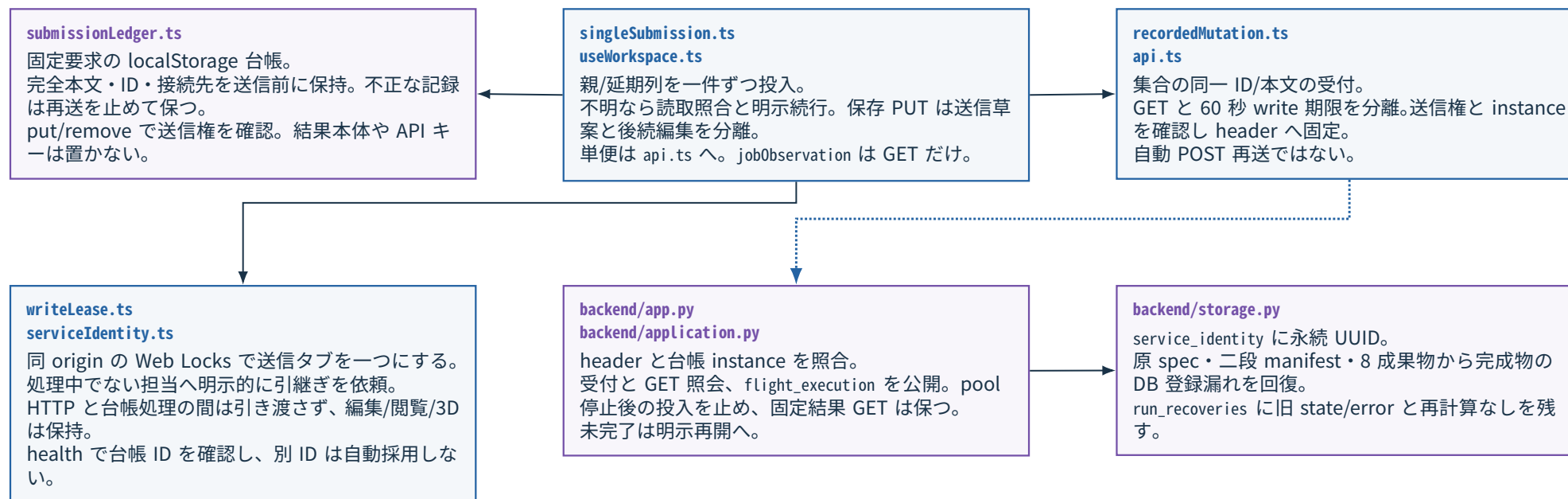
ApplicationService の DB と lock を共有する application の枝。並列計算基盤を追加せず、既存の一飛行を同じ queue へ投入する。



受付と保存先：不明を別の実行へ変えない

階層 3 / frontend の要求台帳 → backend の instance ・ 固定 spec ・ 完成結果

入力草案、送信した要求、受理済みの仕事、表示結果の寿命を分ける。同じ URL であることだけでは、同じ保存台帳とは分らない。



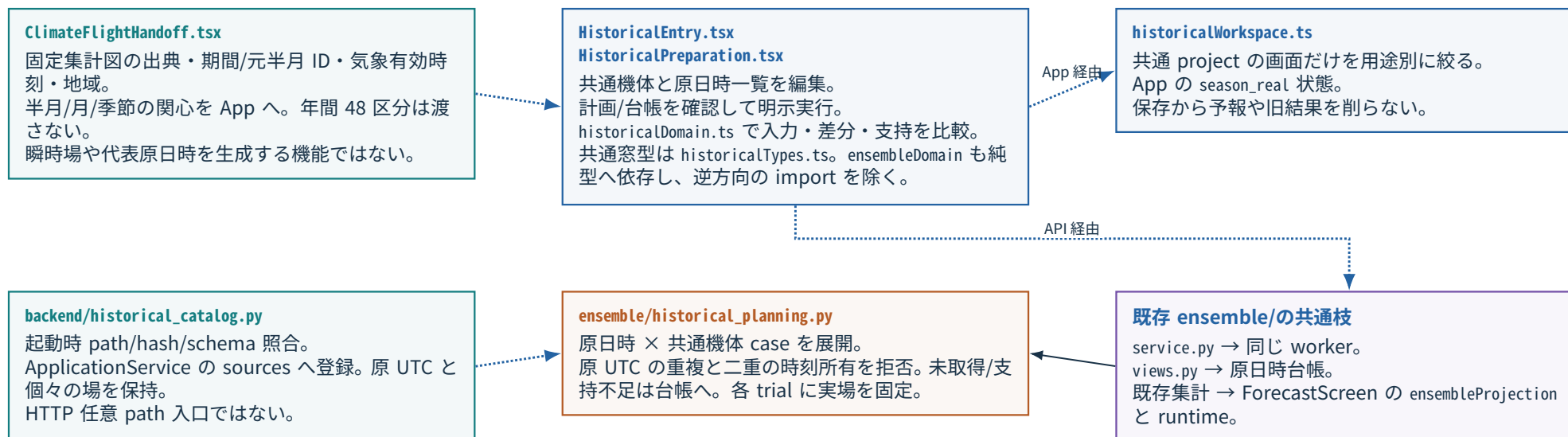
受付・実行・保存の意味を分ける

応答不明は未受付の証明ではない。別 instance へ旧要求を移さない。同一台帳の複製や巻戻し、別ブラウザの排他制御までは保証しない。完成物回復の時刻は回復記録時刻である。操作は COMMANDS C-09、HTTP と例外は実装ノートへ。

原日時：関心から実場を選び、共通核へ渡す

階層 3 / 気象図 → 原日時入力 → historical 計画 → ensemble の共通核

新しい飛行エンジンや別の結果画面は作らない。原日時の所有と場の同一性を明示し、既存の比較・履歴・干渉群へ接続する。



集合 ID と一つの場の ID を混同しない

historical_window_set は窓集合の指紋。支持境界と現在場との照合は members 単位で行う。parameter:null を機体仮幅へ読み替えず、weather_window を原日時台帳へ返す。式・母集団は THEORY、登録と操作は COMMANDS、厳密な保存契約は実装ノートへ。

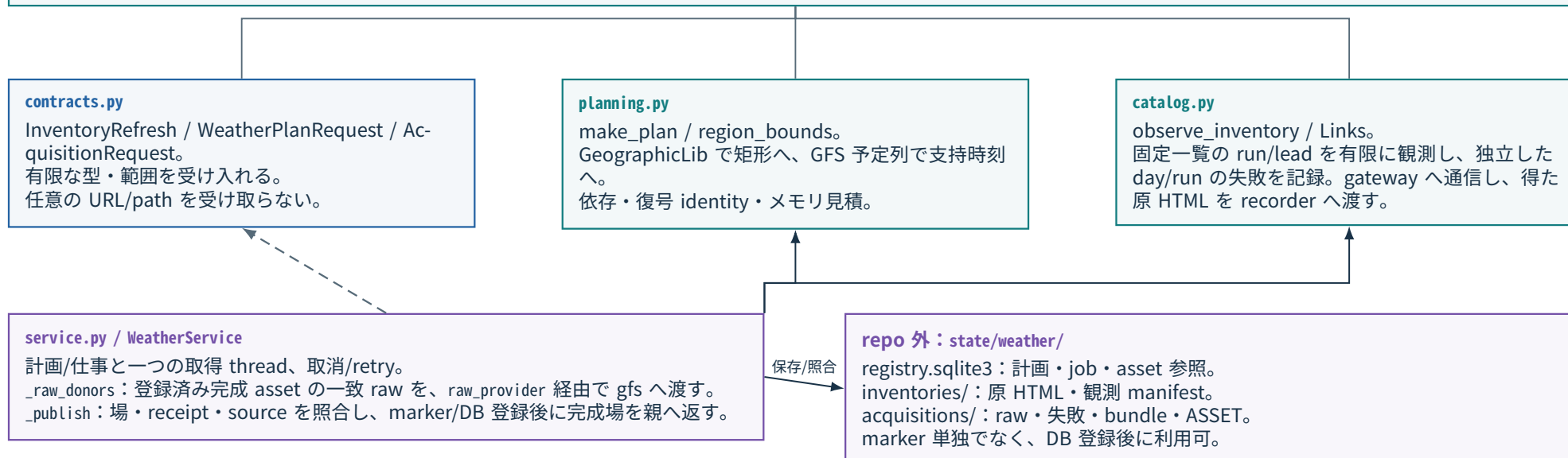
weather : 計画から完成場への公開を管理する

階層 3 / backend → weather → 契約・計画・索引・service

取得順序と保存の寿命を扱う application の枝である。物理式・GRIB の意味・気象場の補間は environment へ委譲する。

backend/weather/ : 固定要求 → 取得 job → 完成 asset

読むもの：観測 inventory、候補ごとの必要窓、領域、予算。渡すもの：保存計画と仕事の状態、検証済み source。候補そのものや過去の run は変更しない。



受付・途中・公開を分ける

ready は取得の受付可。部分 raw は診断/明示 retry 用に保持する。ASSET は partial へ書き、rename 後に hash 照合と DB 登録。完成 asset は再利用しても旧 run を変更しない。
索引の観測・全飛行の支持・科学的精度は別の判断である。

climate : 資料の違いを、固定結果へ結ぶ

階層 3 / backend → climate → service → 二つの資料 adapter

外生 DB と元 UTC 標本を混ぜて一つの資料に見せない。資料別と同じ summary 契約へ変換し、利用者が選んだ結果を別 SQLite へ固定する。

backend/climate/ : ApplicationService.start で起動 → 同期 HTTP route

sources / analyses POST / analysis GET。起動設定だけが DB/manifest path を受ける。利用者 query は source ID ・ 範囲 ・ 気圧面 ・ UTC 時刻。取得用 HTTP と飛行 worker はここから呼ばない。

service.py / ClimateService

sources / create_analysis / get_analysis
複数資料を ID で選び、独立 work lock で集計。
要求/内容の同一性を固定 artifact へ結び、climate.sqlite3 へ短い transaction で保存。
保存 GET は外生資料にも集計 lock にも戻らない。

同じ入口、異なる資料の読取

source.py / ClimateSource : 提供 DuckDB。
保存平均と count の結合。任意の同支持原標本も扱う。
archive.py / WindArchiveSource : 独立した元 UTC 風標本。
両者とも descriptor/query/verify_full を持つ。

contracts.py

Bounds / ClimateQuery
有限値 ・ UTC 集合 ・ 版識別。
全 4 時刻を省略形へ正規化。
HTTP/SQL/保存を持たない。

samples.py

MonthlySamples
manifest/配列/UTC と支持を照合。
完全月の加重経験分位を算出。
両 adapter が呼ぶ。平均/分位から元風を合成しない。

三つの寿命を分ける

外生 DB/配列 : 読取専用。
adapter cache : 有限 ・ process 内。
SQLite artifact : 固定 ・ 再訪可。
図の期間表示切替は再集計しない。

DuckDB は提供 DB、NumPy は原配列と加重経験分位、SQLite は固定結果保存。期間統計は environment.WeatherField を実装しない。相関を持つ飛行場や軌道 MC へ直結したことにはならない。式は THEORY、入力形式/例外/cache は実装ノート、起動/取得は COMMANDS へ。

climate : 時間の結合と、地域の平均を分ける

階層 4 / source → aggregation • periods • extensions → statistics

月や季節への変更は表示用の別母集団を明示する。平均量は件数から、風配は元の階級の件数から作り、架空の同時刻場や分位へ置き換ええない。

source.py : 資料の検査、支持の選択、結果の組立て

DB とコードの指紋・変更を検査し、範囲/時刻の共通集計を有限 cache へ置く。選択面の地域値・風配を合わせ、service.py へ独立した結果を返す。図の粒度変更はこの処理を呼び直さない。

periods.py

periods : 24 半月 → 12 か月 / 4 季節。
元半月 ID、UTC 日数、対応時刻数。
DJF は資料内の 12/1/2 月の pool で、年をまたぐ連続冬列ではない。

aggregation.py

cell_query / annual / spatial
各格子の n 重み時間平均 → Gaussian 地域平均。
件数/格子支持を検査。
R/来向と格子平均の幅を再計算し、scales へ表示尺度を分ける。

extensions.py

inspect_extensions / wind_rose
追加 table の有無と schema を検査。時刻別平均の支持面、単位/キー/件数を確認。
風配は固定 1 地点の階級 count を結合。時刻 subset なら利用不可と返す。

statistics.py

vector_summary / utc_half_months
平均成分・速さ → R/FROM、UTC 暦 → 元の件数。
combine_cells は純粋な加重和による SQL の照合先。
aggregation と periods が使う。

支持と、保存結果の責務

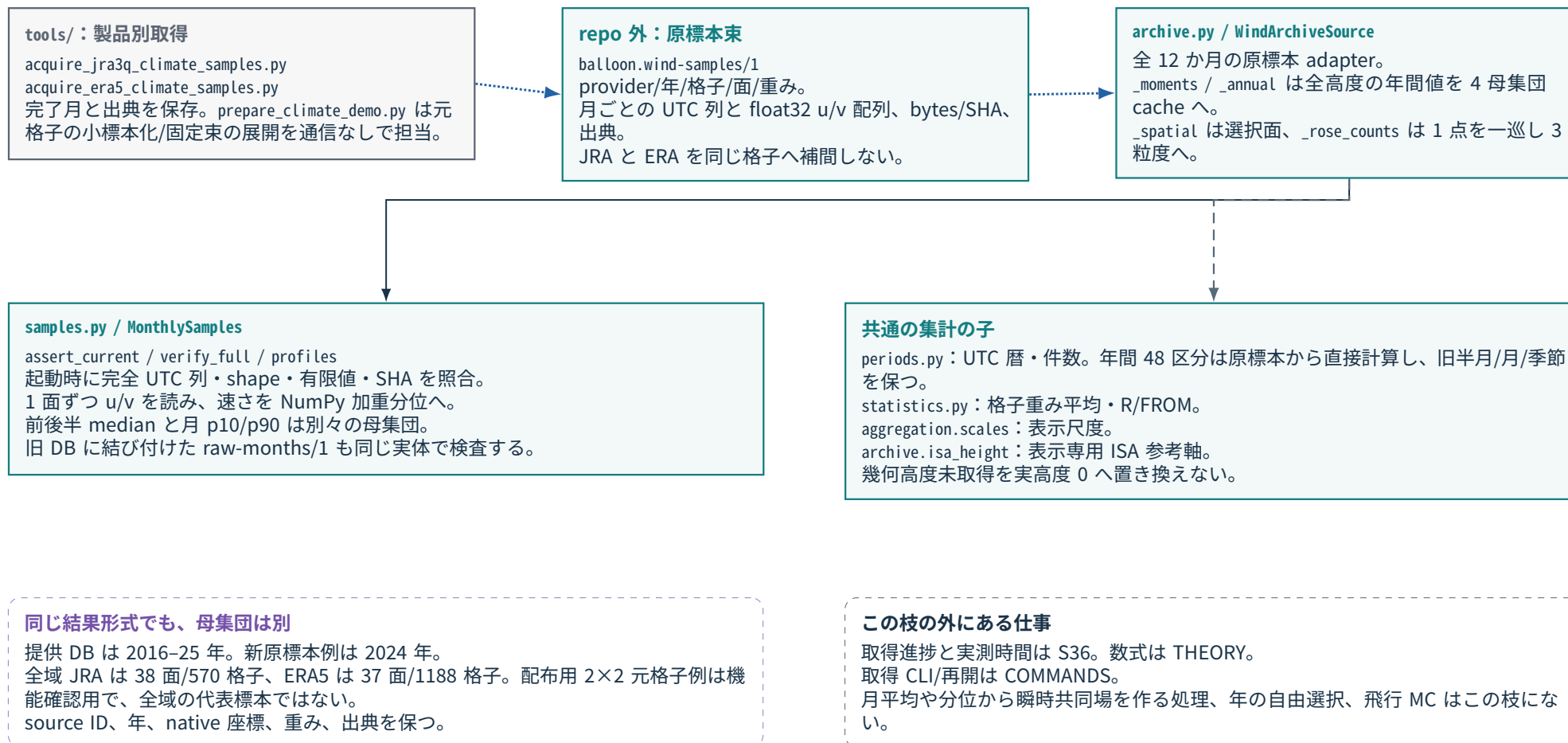
この提供 DB では全時刻 38 面、時刻別 20 面。欠測をゼロ補填しない。風配は地域外でも元地点を明示する。
summaries_by_grain に月/季節を保存。旧 artifact に新集計を後付けしない。

構造の責務はこの頁、n 重み・Gaussian・経験頻度と UTC/JST の意味は THEORY へ。この枝は提供平均と風配 count を結合する。月/地域分位はここで平均せず、同支持の原標本がある場合だけ samples へ渡す。

原標本：取得、検査、分位の担当を分ける

階層 4 / tools → 外生 manifest → archive → samples • periods • statistics

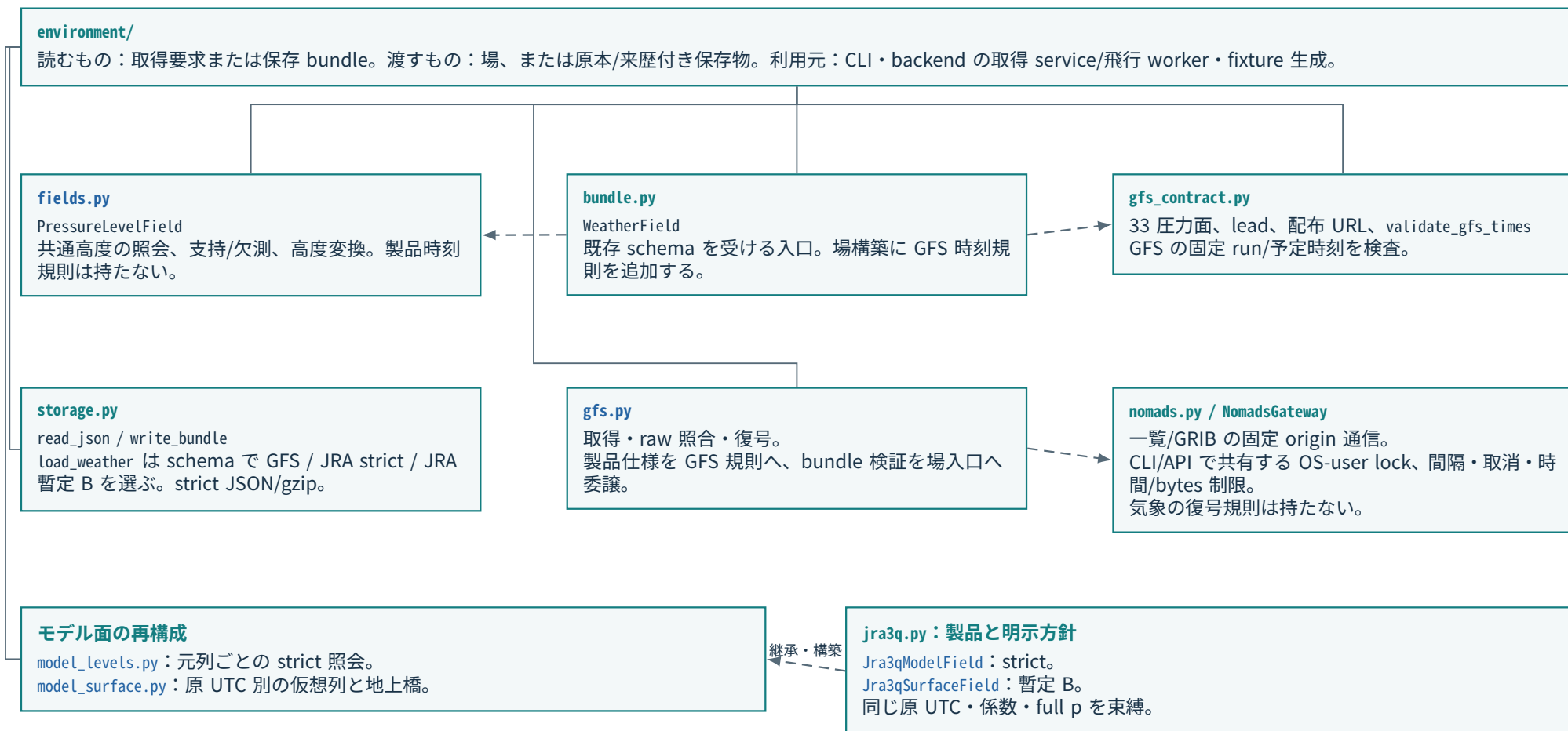
元時刻 u/v の束が初めて、任意の選択格子と UTC 時刻を同じ分布へまとめる根拠になる。取得プログラムはサービスの起動・固定結果保存・画面を所有しない。



environment：製品と場の意味を分ける

階層 2 / balloon_sim → environment → 場・製品・保存・通信

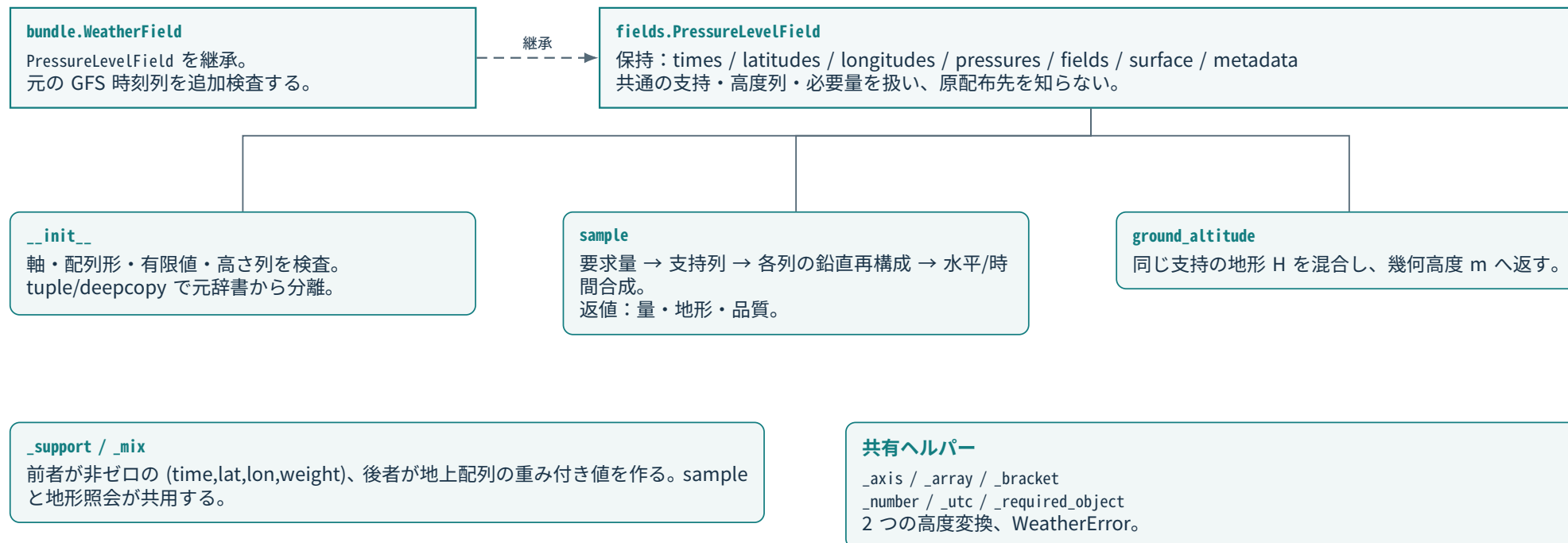
親の責務は「保存入力を、必要な時空間支持を持つ環境として渡すこと」。圧力面/モデル面の再構成と、製品固有の時刻・座標契約を別の実体にする。



圧力面場：構築してから照会する

階層 3 / environment → fields.py / bundle.py → クラスとメソッド

入力の形を受け入れる段階と、一つの時空間点を評価する段階を分ける。値の意味は理論ガイドの気象再構成節へ戻る。



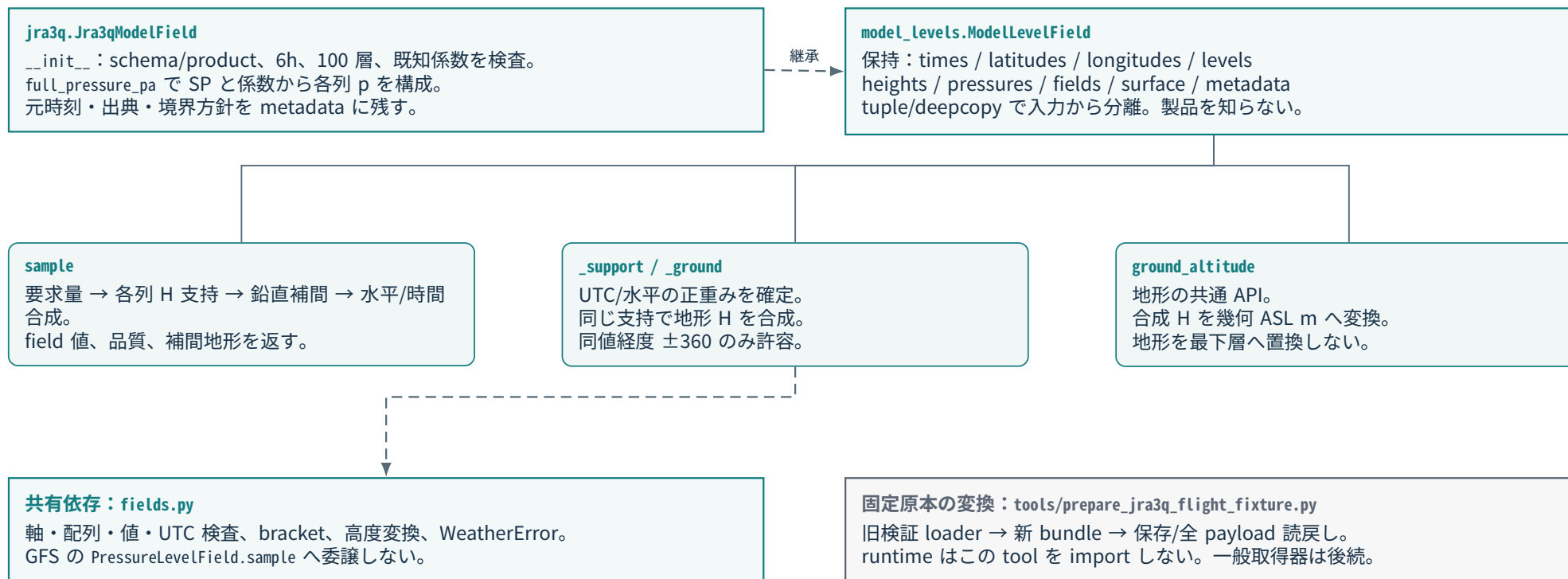
sample 中でのこと、飛行へ渡すこと

列ごとに地下面を除き、地上アンカーまたは有効圧力面から共通高度へ再構成する。その実値を混ぜる。flag/checked/upper はこのメソッドに閉じる小補助。寄与列の地表延長を品質として返し、飛行の相・接地イベントは扱わない。

strict モデル面場：元列の支持を守る

階層 3 / environment → model_levels.py • jra3q.py → クラスとメソッド

飛行核の sample/ground_altitude を共用する。GFS ヘ型を偽装せず、JRA の鉛直座標と厳密支持を明示した枝である。暫定 B でも、この製品検査と原列の保持を土台に使う。

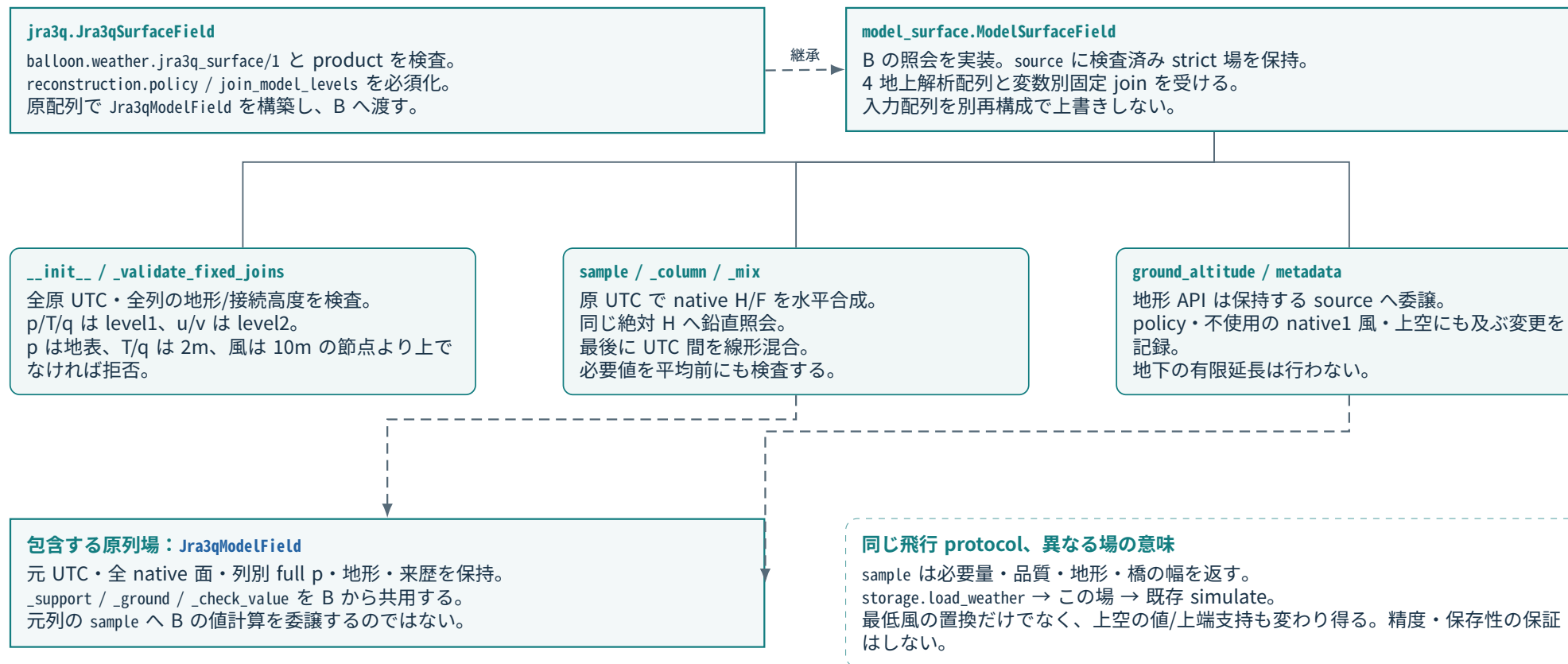


呼出順：CLI → storage の schema dispatch → 同じ flight 核 → 結果保存。保存 B 例の GUI 入口は共通 service へ。任意過去窓の取得画面は未実装。strict では元列の全正重み支持を要求し、地表不足は停止結果になる。地上解析を明示的に使う別枝は次頁の暫定 B。数式は THEORY、旧 strict の操作は COMMANDS C-10 へ。

暫定 B：原 UTC の仮想列から地上へつなぐ

階層 3 / environment → jra3q.py → model_surface.py → 保持する strict 場

旧 strict の sample を書き換えず、別 schema と policy で選ぶ。通信・取得 UI はこの枝に置かない。A の地形列延長は外側の比較実験に残す。

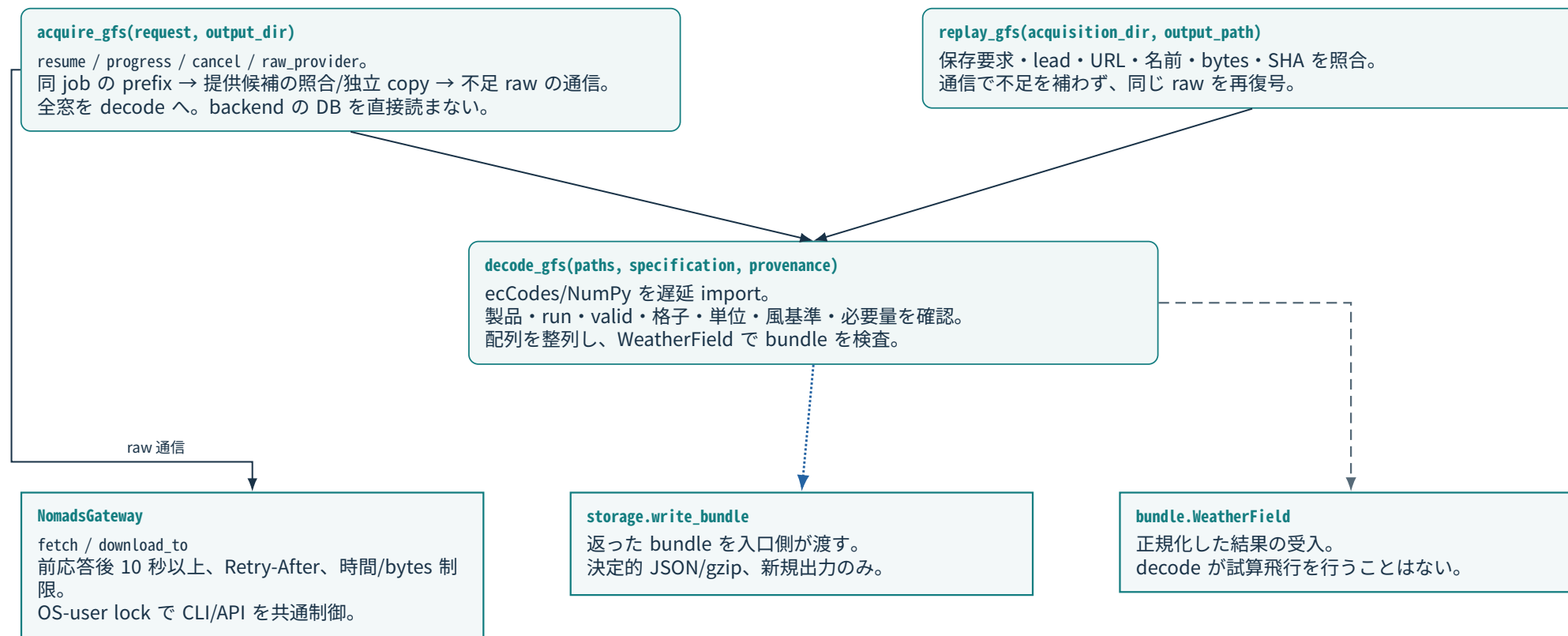


方針の正本は native_horizontal_first_surface_v1。2m/10m 以下の定値、B1 圧力、要求値欠測と仮想列の支持は THEORY へ。保存 schema の自動置換・GFS への fallback・過去場の取得画面は実装していない。

GFS : raw を確定して共通の復号へ渡す

階層 3 / environment → gfs.py → acquire / replay / decode

新規取得・明示 resume と原本再生は、raw の用意と照合で分かれる。復号する量・格子・時刻の意味は同じ decode で受け入れる。



_verified_raw_candidates / _copy_verified_raw は raw の内容とコピー先を照合。候補の登録責務は [weather/service](#) 側に残る。通信は _download → NomadsGateway、時刻/層/URL は gfs_contract、復号見積は estimate_decode_bytes。操作/障害時は COMMANDS・実装ノートへ。

ensemble：元標本と共通集計を、入出力から分ける

階層 2 / balloon_sim → ensemble → sampling・statistics

この枝は純粋な値の変換を担う。抽選した元値を完全設定へ展開し、固定結果を同じ定義で集計する。仕事の投入、DB、HTTP、ファイルの公開は backend 側にある。

balloon_sim/ensemble/：__init__.py が公開名をまとめる

入口は freeze_drawset / resolve_config / analyze_results。一飛行の simulate を内蔵せず、予定標本・個別飛行・表示集計の寿命を分ける。

sampling.py

freeze_drawset：明示した元量・単位・一様幅・理由・seed から、元実現値と draw ID を固定。

resolve_config：選んだ物理元量を一つの config へ展開し、全設定を再検証。

既存 flight/config.py

validate_config / FlightError

展開先の型・モデル・値域を担う。

モデル不活性を入力欄の追加で隠さない。

外部：NumPy

Generator / PCG64 / uniform

乱数列生成は既存ライブラリへ。

seed だけでなく実現値・版を保存側へ返す。

statistics.py

analyze_results：全台帳の ID と選択を検査し、選択群を着地幾何と相別履歴へ渡す。

null 結果や履歴なし停止も、選択母数から消さない。

固定する値 → backend

drawset と完全 config → 計画。

完了結果/選択 ID → 共通分析。

気象の取得や飛行の再実行は、この枝から起こさない。

現在の元量は等温ガス質量・高度破裂・基準降下速度。搭載質量の片側だけの抽選や製品の分布推定は実装していない。方式と式は THEORY、受付予算と API は実装ノートへ。

集合の集計：幾何と時刻の二つの枝へ降りる

階層 3 / ensemble → statistics → landing • histories • geography

読む対象は同じ固定結果と選択 ID である。母数、着地点、相付き記録の違いを、図の描画から独立した共通出力へ残す。

statistics.py / analyze_results

選択 ID・科学結果の外枠を照合 → 選択行を二つの解析へ。選択数、結果あり、履歴あり、欠落を別に返す。run/attempt/source の hash 照合は呼出側が担当する。

landing.py / landing_analysis

着地のみ → 局所平面 → 共分散/固有軸 → 経験 50/90/95% と全点 extent。
点/線/面と非支持理由を返す。含有 ID を保持し、地図の輪郭交差から干渉数を作らない。

geography.py / LocalPlane

xy / lonlat / supported_ring
WGS84 局所方位距離座標と地理座標を相互変換。
GeographicLib を使用。

histories.py / history_analysis

_metric_records で原記録を量と相へ分ける。
同じ経過時刻で相内だけを内挿し、平均/10-90% 帯と各量の有効 N を返す。原 records は変更しない。

外部の一般計算

landing : numpy.linalg.eigh
scipy.spatial.ConvexHull
histories : numpy.interp
quantile / mean。

相と終端の意味

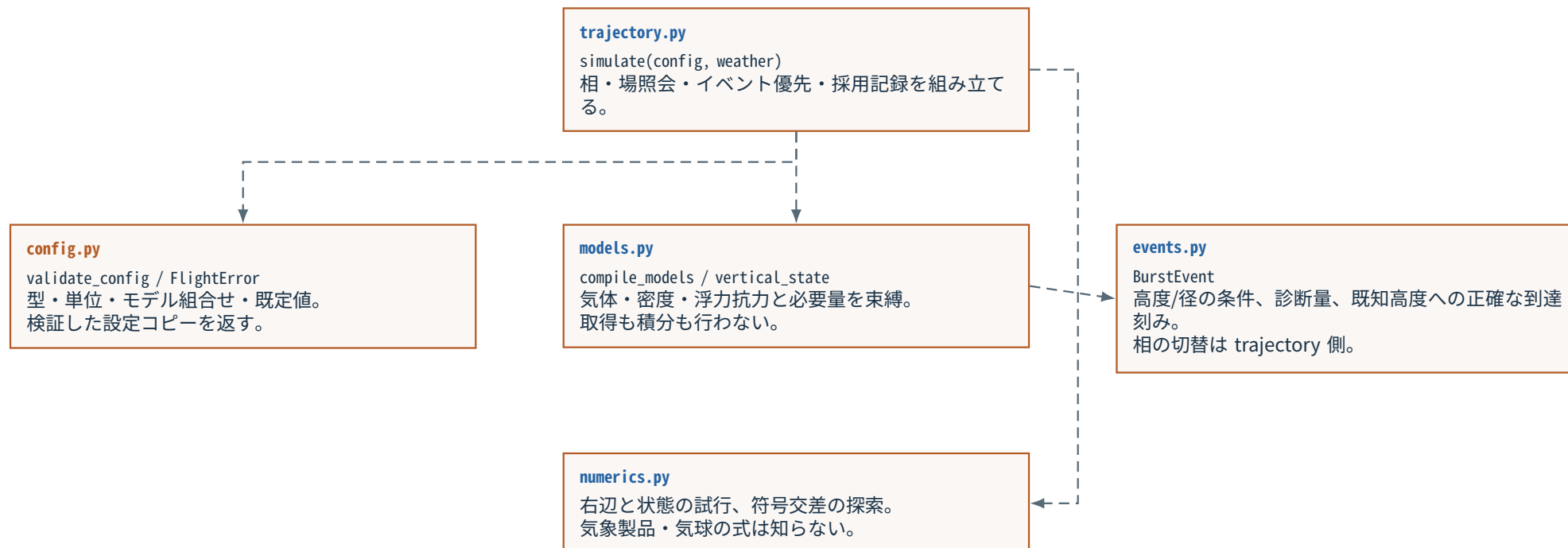
上昇左極限/下降右極限を相別に保持。終端の後へ
外挿しない。
同時刻二相の N は足さない。
平均位置線は実在の一飛行ではない。

局所幾何の非支持は着地そのものの失敗ではない。禁止領域への全着地点判定は frontend/math、選択 ID・領域版・分析 artifact の保存は backend へ戻る。詳しい経験域/内挿の定義と許容差は THEORY に置く。

flight：一飛行の意味を束ねる

階層 2 / balloon_sim → flight → 設定・物理・イベント・数値・進行

親の入力は config と場、出力は一飛行の結果辞書。ここでは取得元やファイル形式を知らず、飛行中の判断に専念する。



モデル追加と数値変更は別の判断

Re 依存は抗力則、慣性や熱は状態と方程式の変更。今回は現物の責務を分離し、任意モデル登録・任意次元の飛行モデルを完成したとはしない。将来の構成は ENV、現在選べる mode は実装ノートと COMMANDS で確認する。

trajectory : 場とモデルを一飛行へ接続する

階層 3 / flight → trajectory.py → simulate と一飛行の局所状態

時刻・相・採用記録は一飛行に属する。数値算術と物理の評価を外へ委譲したうえで、これらの進行を一箇所で判断する。

simulate の進行

設定受入・compile_models → 放球状態 → 試行 → 破裂/地面の先着を選択 → 採用記録 → 相切替または終端。支持を失った場合は、採用済みの結果と停止理由を保つ。

場照会の局所補助

stamp / ground / sample_at
経過秒を UTC へ。必要量をモデルから取得。試行点の地表延長を区別する。

右辺と試行

rhs : 球面移流＋鉛直式
AdaptiveRK45.trial へ委譲
共通の場を積分段ごとに照会。

イベントへの接続

burst_value / locate_crossing
束縛済み BurstEvent へ条件評価、numerics へ根探索を渡す。地面条件は局所 ground。

径破裂で必要な場を照会

採用点と破裂記録

make_record / accept / make_burst_event
make_record の検証後に accept が時刻・状態・記録を一組で採用。到達した破裂点を保ち、下降開始を別に受け入れる。同時刻 2 相を区別。

失敗と共有状態

failure が code・時刻を持つ停止理由へ。
phase / elapsed / state / records / events は simulate の局所状態で、別飛行とは共有しない。

親へ返す結論

降下中の地面交差が landed。上昇衝突・支持喪失・上限超過は stopped。結果辞書を返すまでが責務で、ファイル保存は results。数値法を替える際も、相・停止・採用点の意味を無言で変えない。

物理とイベント：値を評価し、進行は返す

階層 3 / flight → models.py / events.py → 現在の式と必要量

上昇・降下の法則、破裂条件、相を進める操作を分ける。式の導出と仮定は理論ガイドに置き、ここではそれを担う実体を示す。

models.py

読む：検証した config。compile_models が各相の入力を分離。
渡す：FlightModels(ascent,descent,burst)。各相は評価関数と必要量を持つ。

events.py

読む：独立した閾値・gas_state callable・指定上昇速度。
渡す：破裂余裕・診断・正確な刻み・高度丸め。相の設定配置を知らない。

vertical_state

上昇：規定速度 / 等温浮力
降下：CdA / 基準速度
密度を使う経路だけ密度を評価。

_gas_state / _density

前者：体積・直径・投影面積
後者：moist_air_density へ
選んだ仮定を共通に評価する。

BurstEvent

value / diagnostics / exact_step
project_altitude、必要量の有無。
gas_state を引数で受ける。models へ逆 import しない。

注入した気体計算の callable

PhaseModel / FlightModels

compile_models が各相の evaluate と required_fields を束縛する。trajectory は mode 名で式を選ばず、phase と評価契約だけを使う。熱等の状態追加は別の契約拡張が必要。

変更時に一緒に見る境界

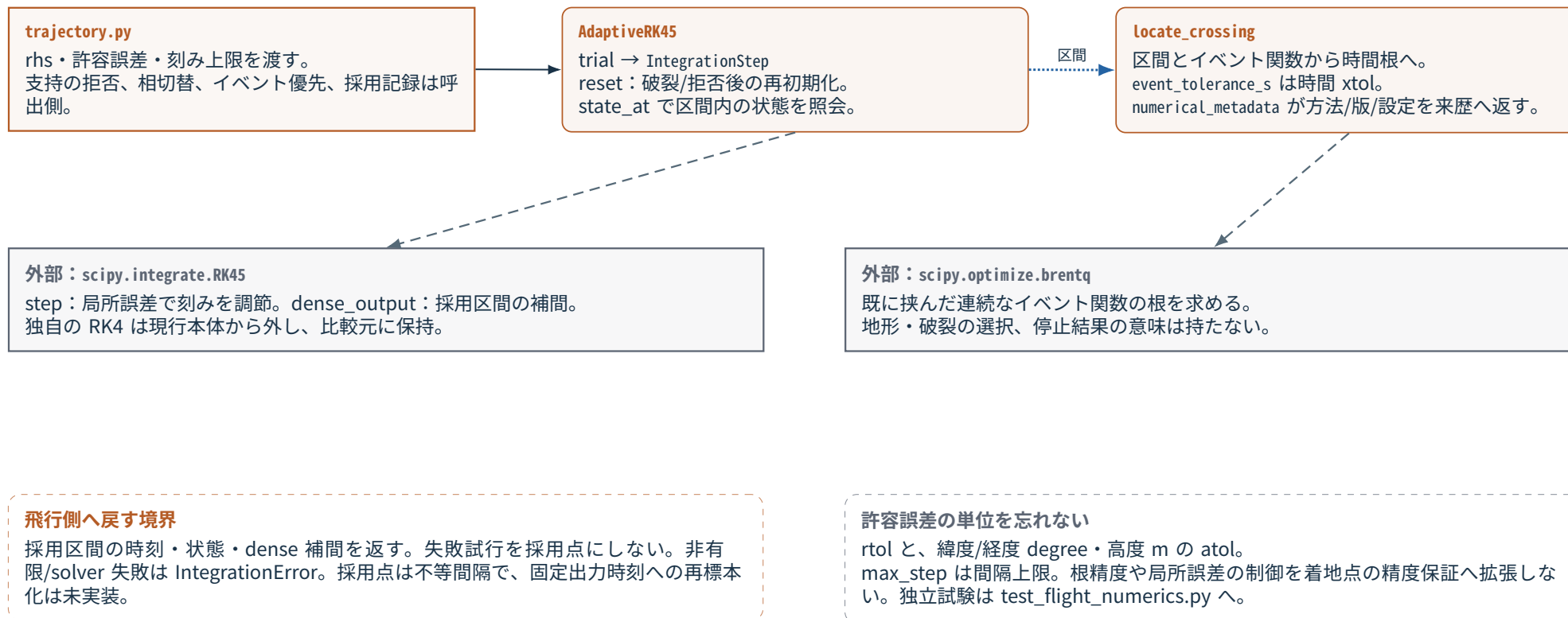
法則の変更 → config と必要量
状態の追加 → trajectory/結果 schema
径の式変更 → 破裂と診断出力
同じ計算自由度を二重に独立入力しない。

現役の単純経路を残して比較する。熱・慣性・Re 依存をまだ選べるようにはしていない。等温仮定の採用条件は後から拡張できる設計であり、現時点の感度試験を再要求しない。

numerics：一般数値法を SciPy へ委譲する

階層 3 / flight → numerics.py → SciPy RK45 と brentq

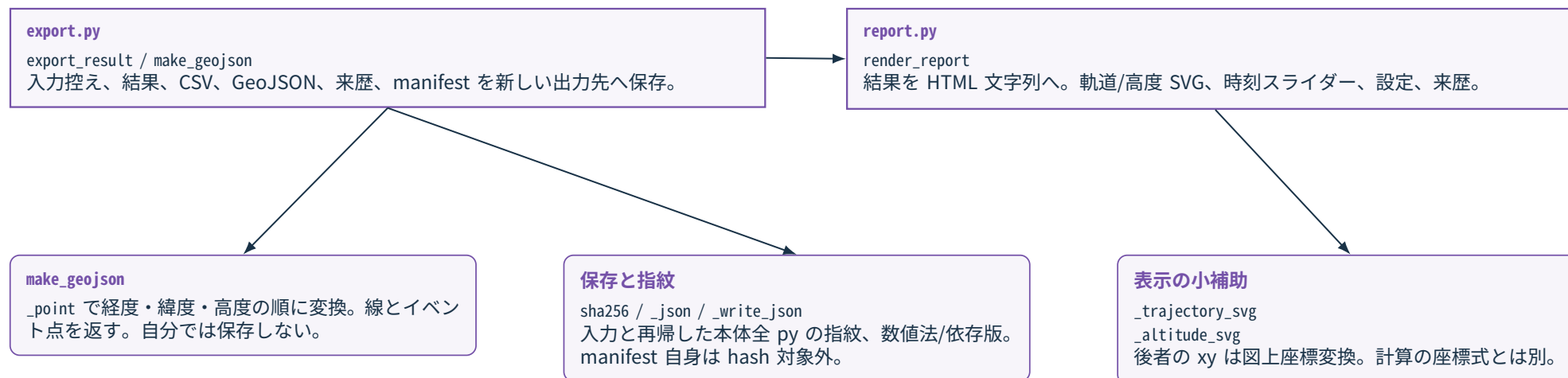
物理・場・相の意味は飛行側が持つ。一般的な誤差推定、適応刻み、区間補間、根探索は既存ライブラリの公開 API へ渡す。



results：意味を変えず、読める結果にする

階層 2 / balloon_sim → results → export.py / report.py

読むものは完了・途中停止を含む結果と入力来歴。ここで物理を再計算したり、停止を成功へ変えたりしない。



表示を変更するときの戻り先

この旧単一 HTML の水平図は背景地図なしの局所投影である。現在の比較地図は screens/forecast の別表示。集合の経験域は ensemble の共通分析から作り、実世界の事故確率とは区別する。単一 HTML の表示を、集合の図と同一の責務にしない。形式と hash の検査、画面の視認、科学的精度は別に評価する。

CLI：利用者の操作を接続する

階層 2 / balloon_sim → cli.py → main

python -m balloon_sim から一つの操作を行う。公開していた整形/保存名は互換再 export で維持し、本体定義は results へ移した。

main(argv=None)

argparse で 4 操作を選択し、入力と既存出力を確認する。呼出結果を stdout へ、エラーを stderr へ、終了コードを呼出元へ返す。

fetch

read_json → acquire_gfs
通信と復号を行う。

replay

replay_gfs
保存 raw を照合し再復号。

inspect

load_weather
場の metadata を表示。

simulate

load_weather + 設定
simulate → export_result

JSON 読取の所在

environment.storage.read_json を直接使い、weather の私有名へ横断しない。旧 cli.read_json の import は同じ実体を再 export する。標準 JSON 以外の暗黙補正をしない。

利用者へ返す意味

0：操作成功、または landed。
2：stopped の部分結果を保存。
1：入力・実行・保存などのエラー。
argparse の usage 終了 2 は別。操作例と失敗後の判断は COMMANDS へ。

検査と道具：何を守るために存在するか

横断枝 / tests と tools → 対象契約と本体への接続

試験数や図の存在だけで構造を受け入れない。守る意味と、その証拠が届く範囲を分ける。

test_flight_weather.py

取得要求、raw、支持/補間/地表。
test_gfs_acquisition.py：共有間隔・通信制限・resume。
test_gfs_raw_reuse.py：候補照合・独立 copy・残量/破損を注入検査。

test_trajectory.py

解析場・独立式・破裂/着地/停止。
flight の変更を守る。
test_flight_numerics.py は SciPy 接続と根を独立検査。

集合の純粋な意味

tests/test_ensemble.py
同じ抽選・方式への展開、点/線/面、経験順位、相/終端・選択母数。
核を試し、実 UI や科学精度の受入とは分ける。

保存標本と JRA 接続

make_flight_fixture.py：保存場から小標本を切り出す。
prepare_jra3q_flight_fixture.py：固定原本を変換。
test_jra3q_flight_field.py：列場・製品契約・保存振分け・飛行停止。
test_jra3q_surface_field.py：B の順序・固定 join・支持/欠測・別 schema。

backend・frontend の試験

backend/tests/test_ensemble.py：固定計画・全台帳・取消/再開・保存。
frontend/src/ensemble.test.ts：参照・描画 adapter の契約。
風統計は test_climate*.py / climate.test.ts。
test_weather_raw_reuse.py：登録完成 asset からの取得。restoration.test.ts：未読参照・寿命/保存境界。

管理・生成

check_state/health/contract
run_checks / prepare_workspace
build_docs / build_guides
計算の実行依存ではない。

変更後は親の用途へ戻る

旧 GFS/JRA 等の試作は比較根拠として保持する。test_flight_cli.py の実場再現・出力署名、backend の実 worker 試験、画面の時刻/相/選択試験は継続する。これらと、原一覽/GRIB を実取得して再閲覧する証拠、資料の通読、科学的な精度評価は別である。取得の一成功から全国・全期間・全モデルの受入へ広げない。

変更したいことから、読む枝を決める

逆引き / 最初の担当 → 横の契約 → 利用と試験への影響

この図で終わらず、該当枝へ降り、最後に全体の目的へ戻る。配置が見つかることと、変更案が妥当なことは別である。

気象の取得・意味を変える

操作/保存：取得 UI → backend/weather

GFS 通信/製品：nomads.py、gfs.py / gfs_contract.py

保存 schema 分岐：storage。GFS 再構成：bundle / fields

JRA 製品/strict：jra3q / model_levels

暫定 B 再構成：model_surface / 明示 policy

支持/品質と全飛行へ影響。根拠は WEATHER / ENV / THEORY。

鉛直則・状態量・破裂を変える

最初：models.py / events.py と config.py

次：trajectory.py の状態・要求量・記録

影響：結果 schema・停止・比較試験

根拠：THEORY / ENV / 実装ノート

積分器を変える

最初：numerics.py の呼出契約

次：trajectory の相・支持・event・出力点

影響：依存/許容誤差/来歴/解析解との比較

採用点/誤差の意味を保ち、一般法は SciPy へ委譲。

操作・結果表示を変える

集合：画面 → 台帳/保存 → 抽選/集計

CLI 単独：cli.py / results/

風統計：資料 adapter/共通図 → climate/

次：草案/固定結果・API・保存版・再閲覧

計算の式を表示層へ複写しない。

根拠：COMMANDS / 実装ノート / CLI 試験

この地図から分かることと、残ること

実パッケージの責務、主要依存、変更の最初の場所と影響先を辿れる。全組合せの物理的採用、全国全期間の取得、実飛行の予測精度を認定する資料ではない。読めない枝が増えたら、図を縮めて押し込む前にコード側の責務を再検討する。